

Dicionários

Dicionários

Dicionários em python também são coleções (assim como lista e tupla). No entanto, não definem nenhum tipo de ordem dentre seus elementos.

Dicionários

Em vez de armazenar os itens por *posição relativa* (como nas listas). Os dicionários realizam um **mapeamento** entre um par **chave/valor**.

Analogia

Para entender melhor o mapeamento, podemos comparar ao uso de um dicionário (*da vida real*). Nele encontramos a relação:

palavra → **significado**.

Analogia

As palavras seriam a **chave** e o **valor** seria a frase que descreve o significado da palavra.

Sintaxe

Podemos declarar dicionários utilizando
`{}`, ou `dict()`.

Syntaxe

```
dicionario1 = {} # dicionário vazio.  
dicionario2 = dict() # dicionário vazio.
```

Sintaxe

Para declarar pares chave/valor utilizamos a estrutura `chave : valor`.

```
diccionario3 = {"nome" : "arthur"}  
diccionario4 = {33 : 44.6}
```


Sintaxe

Os diferentes pares devem ser separados
por ,

```
diccionario = {  
    "nome" : "arthur",  
    "idade": 27  
}
```

Pares Chave/Valor

As vezes os pares **chave/valor** são chamados de **items** ou de **entradas** do dicionário.

Chaves

- Devem ser imutáveis.
- Devem ser únicas dentro do dicionário.
- Podem ser qualquer tipo básico do python (exceto mutáveis).

Chaves

```
nomeDaChave = "nome"
dicionario = {
    (1, 2): 12, # ✓
    1: 1, # ✓
    nomeDaChave: "chave", # ✓
    [3, 4]: 34 # ✗
}
```

Chaves

```
diccionario = {  
    "nome": "arthur", #   
    "nome": "joão" #   
}
```

Mutabilidade

Dicionários podem crescer e diminuir de acordo com a demanda. O valor de uma chave pode ser alterado.

Buscando Items

```
# Acesso direto pela chave  
idade = pessoa["idade"]
```

```
# Busca segura (retorna None ou valor padrão se não existir)  
idade = pessoa.get("idade", 0)
```

Verificando Existência de Chaves

```
peessoa = {"nome": "Arthur", "idade": 27}

if "nome" in peessoa:
    print("A chave 'nome' existe!")

if "email" not in peessoa:
    print("A chave 'email' não existe!")
```


Adicionando Items

```
# Criando ou sobrescrevendo uma chave  
pessoa["cidade"] = "São Paulo"  
  
# múltiplas chaves de uma vez  
pessoa.update({"estado": "SP", "pais": "Brasil"})
```

Removendo Items

```
# Remove a chave e retorna o valor associado  
idade = pessoa.pop("idade")
```

```
# Remove a última chave inserida (retorna o par chave-valor)  
chave, valor = pessoa.popitem()
```

```
# Remove a chave sem retornar o valor  
del pessoa["cidade"]
```

Iterando Sobre Dicionários

```
# Iteração direta sobre chave e valor  
for chave, valor in pessoa.items():  
    print(f"{chave}: {valor}")
```

Iterando Sobre as Chaves

```
# Iterando apenas pelas chaves  
for chave in pessoa.keys():  
    print(chave)
```

Iterando Sobre os Valores

```
# Iterando apenas pelos valores  
for valor in pessoa.values():  
    print(valor)
```

Métodos Úteis

```
# Retorna a quantidade de pares chave/valor  
len(pessoa)  
  
# Cria uma cópia RASA do dicionário  
copia = pessoa.copy()  
  
# Limpa o dicionário  
pessoa.clear()
```

Dicionários Aninhados

Os valores de um dicionário podem ser outros dicionários.

```
alunos = {  
    "2026001": {"nome": "Ana", "nota": 9.5},  
    "2026002": {"nome": "João", "nota": 8.0}  
}
```

Dicionários Aninhados

```
# Acessando dados aninhados  
nome_ana = alunos["2026001"]["nome"]
```